

Raport laboratoryjny (Lista 4)

Przedmiot: Sztuczna Inteligencja i Inżynieria Wiedzy — Lista nr 4

Zakres: Klasyfikacja przeżywalności pacjentów z marskością wątroby — eksploracja danych, przygotowanie, Naiwny Bayes, drzewo decyzyjne, ewaluacja metrykami.

Autor: Dawid Błaszczuk

Spis treści

Raport laboratoryjny (Lista 4).....	1
1. Cel ćwiczenia.....	3
2. Wstęp teoretyczny.....	3
2.1. Uczenie nadzorowane i podział danych.....	3
2.2. Naiwny Klasyfikator Bayesa	3
2.3. Drzewo decyzyjne.....	3
2.4. PCA i metryki	3
3. Zadanie 1 — Eksploracja danych (10 pkt).....	4
3.1. Źródło i wczytanie	4
3.2. Struktura zbioru.....	4
3.3. Brakujące wartości.....	4
3.4. Rozkład klasy docelowej (<code>Status</code>)	5
3.5. Statystyki opisowe	5
3.6. Wnioski z eksploracji.....	5
4. Zadanie 2 — Przygotowanie danych (30 pkt)	6
4.1. Podział train / test.....	6
4.2. Obsługa brakujących wartości (imputacja).....	6
4.3. Warianty przetwarzania	6
4.4. Oczekiwany wpływ na klasyfikację.....	6
5. Zadanie 3 — Klasyfikacja (40 pkt + bonus).....	7
5.1. Metodologia eksperymentów	7
5.2. Naiwny Klasyfikator Bayesa — hiperparametry.....	7
5.3. Drzewo decyzyjne — hiperparametry	7
5.4. Bonus — Random Forest.....	8
5.5. Bonus — łagodzenie przeuczenia (drzewo decyzyjne)	8
6. Zadanie 4 — Ewaluacja i interpretacja (20 pkt)	9
6.1. Tabela zbiorcza wyników	9
6.2. Szczegółowa analiza błędów (Confusion Matrix)	9
6.3. Dodatkowy eksperyment: <code>class_weight='balanced'</code>	10
6.4. Ranking konfiguracji pod kątem wykrywania klasy <code>CL</code>	10
6.5. Interpretacja metryk.....	11
7. Podsumowanie i wnioski	12
8. Biblioteki.....	12

1. Cel ćwiczenia

Celem listy jest praktyczne zapoznanie z prostymi algorytmami uczenia maszynowego oraz typowymi krokami projektu ML:

- eksploracja danych i sformułowanie problemu klasyfikacji,
- przygotowanie danych (podział, imputacja braków, transformacje),
- eksperymenty z **Naiwnym Klasyfikatorem Bayesa** i **drzewem decyzyjnym** przy różnych hiperparametrach,
- ocena wyników metrykami klasyfikacji i ich interpretacja.

Problem: **prognozowanie statusu pacjenta** (*Status*) na podstawie 17 cech klinicznych ze zbioru *Cirrhosis Patient Survival Prediction* (UCI, 418 obserwacji, badanie Mayo Clinic 1974–1984).

2. Wstęp teoretyczny

2.1. Uczenie nadzorowane i podział danych

W zadaniu stosujemy **uczenie nadzorowane** — model uczy się mapowania cech na etykietę *Status*. Dane dzielimy na:

- **zbiór uczący** (80%) — dopasowanie modelu i transformacji,
- **zbiór testowy** (20%) — niezależna ocena jakości.

2.2. Naiwny Klasyfikator Bayesa

Klasyfikator probabilistyczny zakładający **niezależność cech** przy danej klasie. W implementacji: *GaussianNB* (scikit-learn) — ciągłe cechy modelowane rozkładami Gaussa; hiperparametr *var_smoothing* stabilizuje wariancję.

2.3. Drzewo decyzyjne

Hierarchiczny podział przestrzeni cech (kryterium **Gini** lub **entropia**). Hiperparametry: *max_depth*, *criterion*. Drzewa są podatne na **przeuczenie** — ograniczenie głębokości (*max_depth=3*) redukuje dopasowanie do szumu treningowego.

2.4. PCA i metryki

PCA redukuje wymiarowość cech numerycznych. Do oceny klasyfikacji wieloklasowej użyto średnich **makro**: accuracy, precision, recall, F1.

3. Zadanie 1 — Eksploracja danych (10 pkt)

3.1. Źródło i wczytanie

Dane pobrano z repozytorium UCI przez `ucimlrepo(fetch_ucirepo(id=878))`, następnie połączono cechy i etykietę w jedną ramkę `df` (418 wierszy $\times$ 18 kolumn).

3.2. Struktura zbioru

Element	Wartość
Liczba obserwacji	418
Liczba cech	17 (+ kolumna docelowa <code>Status</code>)
Typy	10 kolumn numerycznych (<code>float64/int64</code>), 7 kategorycznych (<code>str</code>)

3.3. Brakujące wartości

Najwięcej braków w kolumnach kategorycznych związanych z protokołem leczenia oraz w pomiarach laboratoryjnych:

Kolumna	Liczba braków (z 418)
Tryglicerides	136
Cholesterol	134
Copper	108
Alk_Phos, SGOT	106 każda
Drug, Ascites, Hepatomegaly, Spiders	105 każda
Platelets	11
Stage	6
Prothrombin	2
Age, Sex, Edema, Bilirubin, Albumin, Status	0

3.4. Rozkład klasy docelowej (Status)

Klasa	Znaczenie (wg listy)	Liczność	Udział
C	Ocenzurowana (wynik inny niż śmierć z powodu choroby)	232	~55,5%
D	Śmierć	161	~38,5%
CL	Ocenzurowana z powodu przeszczepu wątroby	25	~6,0%

3.5. Statystyki opisowe

- **Age:** średnio 18 533 dni (50,8 roku); zakres 9 598–28 650 dni.
- **Bilirubin:** średnia 3,22 mg/dl, duży rozrzut (max 28) — typowa zmienna prognostyczna w marskości.
- **Alk_Phos, SGOT:** wysokie odchylenia standardowe i skrajne wartości (np. Alk_Phos max 13 862) — cechy wymagające skalowania lub redukcji wymiaru.
- **Stage:** mediana 3, większość pacjentów w zaawansowanych stadiach histologicznych (3–4).

3.6. Wnioski z eksploracji

1. Zbiór jest **niewielki** i **niezbalansowany** w klasie CL.
2. Braki są **rozproszone** — sensowna jest **imputacja** zamiast usuwania wierszy (zachowanie 418 próbek).
3. Cechy numeryczne mają **różne skale** — standaryzacja przed NB/PCA jest uzasadniona.
4. Cechy kategoryczne wymagają **kodowania** przed algorytmami opartymi o rozkłady ciągłe.

4. Zadanie 2 — Przygotowanie danych (30 pkt)

4.1. Podział train / test

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

Zbiór	Liczba pacjentów
Treningowy	334
Testowy	84

Stratyfikacja utrzymuje proporcje klas w obu podzbiorach.

4.2. Obsługa brakujących wartości (imputacja)

Zastosowano `SimpleImputer` w pipeline `ColumnTransformer`:

Typ cech	Strategia	Uzasadnienie
Numeryczne	median	odporność na wartości odstające
Kategoryczne	most_frequent	zachowanie dominujących kategorii

Dodatkowo kategorie kodowano `OrdinalEncoder`

(`handle_unknown='use_encoded_value', unknown_value=-1`).

Weryfikacja: w zbiorze treningowym 742 braków przed transformacją → 0 po imputacji i kodowaniu.

4.3. Warianty przetwarzania

Przygotowano trzy reprezentacje danych do eksperymentów:

Wariant	Opis
Dane czyste	Po imputacji i kodowaniu (<code>X_train_clean</code> , <code>X_test_clean</code>) — bez skalowania
Dane standaryzowane	<code>StandardScaler</code> na kolumnach numerycznych
Dane PCA	5 składowych głównych z cech numerycznych (po standaryzacji) + zachowane kolumny kategoryczne

4.4. Oczekiwany wpływ na klasyfikację

- **Standaryzacja** — istotna dla `GaussianNB`.
- **PCA** — może uprościć drzewa, ale utrudnia interpretację pojedynczych biomarkerów.
- **Brak skalowania** — często wystarczający dla drzew.

5. Zadanie 3 — Klasyfikacja (40 pkt + bonus)

5.1. Metodologia eksperymentów

Dla każdej konfiguracji: dopasowanie na zbiorze treningowym, predykcja na **zbiorze testowym** (84 próbki), zapis metryk makro. Funkcja pomocnicza `ocen_i_zapisz + sklearn.base.clone` zapewniają powtarzalność.

5.2. Naiwny Klasyfikator Bayesa — hiperparametry

Testowano ``var_smoothing`` $\in \{1e-9, 1e-5, 0.1\} \times 3$ warianty danych = 9 uruchomień.

<code>var_smoothing</code>	Dane czyste	Standaryzowane	PCA
1e-9	Acc 0,679	0,321	0,119
1e-5	0,667	0,595	0,417
0,1	0,571	0,679	0,679

Wnioski: NB najlepiej działa na **danych czystych** lub **standaryzowanych/PCA** przy większym wygładzaniu wariancji; **standaryzacja z małym smoothing (1e-9)** drastycznie pogarsza wynik (0,321) — cechy po skalowaniu wymagają innej stabilizacji wariancji.

5.3. Drzewo decyzyjne — hiperparametry

Trzy konfiguracje $\times 3$ warianty danych = 9 uruchomień:

Konfiguracja	<code>`max_depth`</code>	<code>`criterion`</code>
Głębokie	<code>None</code>	Gini
Regularyzowane	3	Gini
Entropia	7	Entropy

Najlepszy wynik w całej tabeli: Decision Tree - Regularyzowane (`max_depth=3`) [Dane PCA] — **Accuracy 0,738**, F1-macro 0,497.

Konfiguracja drzewa	Najlepszy Acc (wariant danych)
Regularyzowane (depth=3)	0,738 (PCA)
Głębokie (depth=None)	0,655 (czyste / standaryzowane)
Entropia (depth=7)	0,643 (PCA)

Głębokie drzewo na danych PCA: Acc spada do 0,571 — przeuczenie / niestabilność na zredukowanych cechach.

5.4. Bonus — Random Forest

`RandomForestClassifier(n_estimators=100, max_depth=5)` na danych standaryzowanych:

Metryka	Wartość
Accuracy	0,726
Precision (macro)	0,488
Recall (macro)	0,502
F1 (macro)	0,492

RF poprawia wynik względem pojedynczego drzewa głębokiego (0,655), ale **nie przewyższa** najlepszego drzewa regularyzowanego na PCA (0,738).

5.5. Bonus — łagodzenie przeuczenia (drzewo decyzyjne)

Porównanie dopasowania na **danych czystych**:

Model	Accuracy (trening)	Accuracy (test)
<code>max_depth=None</code>	1,000	0,655
<code>max_depth=3</code>	0,763	0,655

Ograniczenie głębokości **obniża dopasowanie do treningu** ($1,0 \rightarrow 0,763$) przy **tej samej jakości na teście** — klasyczny objaw przeuczenia głębokiego drzewa na małym zbiorze. Model prostszy jest preferowany.

6. Zadanie 4 — Ewaluacja i interpretacja (20 pkt)

6.1. Tabela zbiorcza wyników

Pełna tabela wygenerowana w notebooku. Poniżej przedstawiam skrót skonfigurowany z **Accuracy** $\geq 0,65$ na zbiorze testowym:

Model i wariant danych	Accuracy	Prec. (macro)	Recall (macro)	F1 (macro)
DT Regularyzowane (depth=3) [PCA]	0,738	0,498	0,506	0,497
Random Forest [standaryzowane]	0,726	0,488	0,502	0,492
Naive Bayes (1e-9) [czyste]	0,679	0,530	0,514	0,510
Naive Bayes (0,1) [standaryzowane]	0,679	0,480	0,454	0,452
Naive Bayes (0,1) [PCA]	0,679	0,498	0,451	0,449
DT Głębokie [czyste / standaryzowane]	0,655	0,513	0,513	0,506
DT Regularyzowane (depth=3) [czyste / standaryz.]	0,655	0,456	0,443	0,442
Naive Bayes (1e-5) [czyste]	0,667	0,503	0,434	0,424

6.2. Szczegółowa analiza błędów (Confusion Matrix)

Dla najlepszego modelu (Decision Tree, max_depth=3, Dane PCA) otrzymano następującą macierz pomyłek:

Rzeczywista \ Predykcja	pred_C	pred_D	pred_CL
true_C	42	5	0
true_D	12	20	0
true_CL	4	1	0

Kluczowe liczby dla klasy mniejszościowej CL:

- CL poprawnie wykryte ($cm[2, 2]$): **0**
- CL pomyłone z C ($cm[2, 0]$): **4**
- CL pomyłone z D ($cm[2, 1]$): **1**

Wniosek: mimo dobrej accuracy globalnej model nie nauczył się rozpoznawać klasy CL i wszystkie przypadki tej klasy przypisał do klas dominujących (C lub D).

6.3. Dodatkowy eksperyment: `class_weight='balanced'`

Aby poprawić wykrywanie klasy mniejszościowej CL, wykonano dodatkowy eksperyment:

```
DecisionTreeClassifier(max_depth=3, criterion='gini',  
class_weight='balanced', random_state=42)
```

Wyniki dla klasy CL:

- CL poprawnie wykryte (`cm[2, 2]`): **1**
- CL pomyłone z C (`cm[2, 0]`): **3**
- CL pomyłone z D (`cm[2, 1]`): **1**

Wniosek: użycie wag klas poprawiło wykrywanie CL (z 0 do 1 poprawnej predykcji), ale nadal większość przypadków tej klasy jest mylona z klasami dominującymi. To potwierdza, że sam `class_weight='balanced'` pomaga tylko częściowo i warto dalej testować inne rozwiązania problemu.

6.4. Ranking konfiguracji pod kątem wykrywania klasy CL

W dodatkowym benchmarku porównano wszystkie badane modele (`GaussianNB`, różne warianty drzew, `RandomForest`, wersje `balanced`) na trzech reprezentacjach danych. Oprócz liczby poprawnie wykrytych CL analizowano też, ile razy model w ogóle przewidział klasę CL.

Najważniejsze obserwacje:

- `GaussianNB` ($1e-9$) na danych standaryzowanych i PCA wykrywa **5/5** przypadków CL (`cm[2, 2]=5`), ale jednocześnie przewiduje CL bardzo często: odpowiednio **51** i **78** razy.
- To oznacza bardzo dużo fałszywych alarmów (`pred_CL` dla próbek rzeczywiście C lub D), więc sam wynik `cm[2, 2]` nie może być jedynym kryterium wyboru modelu.
- `DecisionTree` (`max_depth=3`, `class_weight='balanced'`) daje skromniejszy, ale bardziej konserwatywny profil: `cm[2, 2]=1`, `cm[2, 0]=3`, `cm[2, 1]=1`.
- Dla wielu konfiguracji (`RandomForest`, część drzew i NB z innymi parametrami) `cm[2, 2]=0`, czyli brak poprawnej detekcji klasy mniejszościowej.

Wniosek praktyczny: przy ocenie klasy CL trzeba równoważyć **recall klasy mniejszościowej** z **precyzją predykcji `CL`**. Modele „łapiące wszystko jako CL” poprawiają recall, ale pogarszają użyteczność kliniczną.

6.5. Interpretacja metryk

Accuracy (~66–74%) — przy dominacji klasy C model może osiągać przyzwoitą trafność, przewidując głównie C i D. Accuracy **nie wystarcza** do oceny jakości dla klasy CL (25 przypadków w całym zbiorze, ~5 w teście).

Precision / Recall / F1 (macro) — uśrednienie po klasach bez wag licznosci; wartości **0,43–0,53** wskazują na **słabsze rozpoznawanie klasy mniejszościowej** i nierówną jakość między klasami.

Porównanie przygotowania danych:

Transformacja	Typowy efekt w eksperymencie
Czyste	Dobre dla NB i DT bez PCA; interpretowalne progi
Standaryzowane	Poprawia NB przy <code>var_smoothing=0,1</code> ; RF najlepszy w tej reprezentacji
PCA	Najlepsze dla regularyzowanego DT ; pogarsza NB przy małym smoothing

Dlaczego PCA pomogło drzewu (`max_depth=3`)? W tym zadaniu PCA prawdopodobnie zadziałało jako reduktor szumu i współliniowości między cechami laboratoryjnymi. Dla płytkiego drzewa oznacza to prostsze, stabilniejsze podziały i mniejsze ryzyko „błądzenia” po mało istotnych cechach.

Komentarz do zapaści Naive Bayes (`Acc = 0,321` dla danych standaryzowanych i `var_smoothing=1e-9`): po standaryzacji część rozkładów klasowych może mieć bardzo małą wariancję (szczególnie dla klasy rzadkiej CL). Przy bardzo małym wygładzaniu model staje się numerycznie niestabilny (silnie „ostre” gęstości Gaussa), co zniekształca prawdopodobieństwa a posteriori i pogarsza klasyfikację.

Porównanie klasyfikatorów:

- **Drzewo regularyzowane + PCA** — najwyższa accuracy w projekcie.
- **Random Forest** — stabilna alternatywa, mniej podatna na pojedynczy podział losowy niż głębokie drzewo.
- **GaussianNB** — prosty baseline; wrażliwy na skalowanie i `var_smoothing`.

7. Podsumowanie i wnioski

1. Problem **klasyfikacji 3-klasowej** przeżywalności w marskości wątroby został zrealizowany zgodnie ze schematem listy: EDA → imputacja → transformacje → NB i drzewa → metryki.
2. **Imputacja medianą / modą** pozwoliła wykorzystać wszystkie 418 rekordów i usunęła 742 braki w podzbiorze treningowym.
3. **Najlepsza konfiguracja** w przeprowadzonych testach: drzewo decyzyjne z `max_depth=3` na cechach po **PCA** (Acc test = **0,738**).
4. **Przeuczenie drzewa** potwierdzono porównaniem `Acc_train=1,0` vs `Acc_test=0,655`; regularyzacja głębokości obniża dopasowanie treningowe bez pogorszenia testu.
5. Klasa **CL** pozostaje trudna — część modeli wykrywa więcej przypadków **CL**, ale kosztem wielu fałszywych alarmów; dlatego w dalszej pracy warto stroić modele pod kompromis precision/recall dla klasy mniejszościowej.

8. Biblioteki

Biblioteka	Zastosowanie
<code>pandas</code>	Ramki danych, eksploracja, tabele wyników
<code>ucimlrepo</code>	Pobranie zbioru UCI (id=878)
<code>scikit-learn</code>	<code>train_test_split</code> , imputacja, skalowanie, PCA, klasyfikatory, metryki
<code>sklearn.compose.ColumnTransformer</code>	Osobne pipeline dla cech numerycznych i kategorycznych
<code>sklearn.pipeline.Pipeline</code>	Imputacja + kodowanie w jednym kroku
<code>sklearn.preprocessing.OrdinalEncoder</code> , <code>StandardScaler</code>	Kodowanie i standaryzacja
<code>sklearn.decomposition.PCA</code>	Redukcja wymiaru (5 składowych)
<code>sklearn.naive_bayes.GaussianNB</code>	Naiwny Bayes
<code>sklearn.tree.DecisionTreeClassifier</code>	Drzewo decyzyjne
<code>sklearn.ensemble.RandomForestClassifier</code>	Bonus — las losowy
<code>sklearn.metrics</code>	<code>accuracy_score</code> , <code>precision_score</code> , <code>recall_score</code> , <code>f1_score</code>